

Python Backend — 60-Day Job Action Checklist

21 Sep 2026 → 19 Nov 2026 | Applications start Day 26 | Learning continues throughout

Daily rule: 3 focused hours. Before Day 26, prioritize learning + building. From Day 26 onward, combine learning, project work, applications, and interview preparation. No roadmap can guarantee employment; this checklist is designed to maximize job-readiness and interview opportunities.

Day 1 — Python Basics

- Install/verify Python + VS Code
- Variables, data types, input/output
- Type conversion and operators
- Code: personal information + salary calculator
- Save code and make first Git/local checkpoint

Day 2 — Conditions

- if / elif / else
- Comparison and logical operators
- Truth values and Boolean logic
- Build eligibility/grade calculator
- Solve 5 small condition problems

Day 3 — Loops

- for / while / range
- break / continue
- Nested loops basics
- Build number-analysis program
- Solve 5 loop problems

Day 4 — Strings

- Indexing and slicing
- Common string methods
- f-strings
- Build text analyzer
- Solve 5 string problems

Day 5 — Collections I

- Lists
- Tuples
- Sets
- Mutability basics
- Build student marks manager
- Practice 5 collection problems

Day 6 — Collections II

- Dictionaries
- Nested dictionaries
- Dictionary methods
- Comprehensions
- Build contact manager
- Practice 5 dictionary problems

Day 7 — Functions

- Parameters and arguments
- return values
- Scope
- Reusable functions
- Refactor previous mini-projects into functions
- Write 5 functions from memory

Day 8 — Files + JSON

- File open/read/write
- CSV basics
- JSON load/dump
- Add file persistence to Expense Tracker
- Test save/load behavior

Day 9 — Exceptions

- try / except / finally
- Common exceptions
- Custom error messages
- Add robust error handling to Expense Tracker
- Test invalid inputs

Day 10 — Modules + Environment

- Modules and imports
- Packages
- venv
- pip
- requirements.txt
- Create clean Expense Tracker project structure

Day 11 — OOP I

- Classes and objects
- `__init__`
- Instance methods
- Encapsulation basics
- Refactor Expense Tracker around classes

Day 12 — OOP II

- Inheritance
- Polymorphism
- Composition
- Modules/packages review
- Clean up project architecture

Day 13 — Git/GitHub + SQL Start

- git init/status/add/commit
- Create GitHub repository
- Write README
- SQL SELECT/WHERE/ORDER BY/LIMIT
- Commit and push project

Day 14 — SQL CRUD

- INSERT / UPDATE / DELETE
- Aggregations: COUNT/SUM/AVG
- GROUP BY / HAVING
- Practice 10–15 SQL queries
- Document SQL notes

Day 15 — SQL Joins

- INNER JOIN
- LEFT JOIN
- Primary/foreign keys
- Practice 15–20 join/aggregation queries
- Design Expense Tracker relational model

Day 16 — PostgreSQL

- Install/use PostgreSQL
- Create database/tables
- Data types and constraints
- PK/FK
- Create users/expenses/categories/budgets tables

Day 17 — Database Practice

- Insert sample data
- CRUD in PostgreSQL
- Joins and aggregation on project data
- Basic indexes
- Practice 10 database queries

Day 18 — Python + PostgreSQL

- Connect Python to PostgreSQL
- Review sessions/connections concept
- Test CRUD from Python

- Move Expense Tracker data from JSON toward DB

Day 19 — HTTP Fundamentals

- Request/response
- JSON
- Headers
- Path/query parameters
- HTTP status codes
- Use Postman/Swagger-style API testing

Day 20 — REST APIs

- GET/POST/PUT/PATCH/DELETE
- REST resource design
- Build a tiny CRUD API
- Test endpoints
- Write request/response examples

Day 21 — FastAPI I

- Install FastAPI + server
- Routes
- GET/POST
- Swagger/OpenAPI
- Create Expense API skeleton

Day 22 — FastAPI II + Pydantic

- Request bodies
- Response models
- Validation
- Path/query parameters
- Error handling
- Implement expense endpoints

Day 23 — SQLAlchemy

- Engine/session concepts
- Models
- CRUD
- Relationships
- Connect FastAPI → SQLAlchemy → PostgreSQL

Day 24 — JWT Authentication

- Password hashing
- Register/login
- JWT access token
- Protected routes
- User-specific expense authorization

Day 25 — MVP + Application Prep

- Finish core Expense API CRUD
- Add filtering/category/monthly total if possible
- Test registration/login/protected routes
- Clean README + GitHub
- Create resume project entry
- Prepare application tracker
- Prepare list of 20–30 target roles
- Tomorrow: begin applications

Day 26 — Applications Start

- Apply to 5–10 relevant roles
- Track company/role/date/status
- Continue Expense API improvements
- Review one Python interview topic
- Do not stop learning

Day 27 — Auth Hardening

- Review JWT flow
- Verify protected endpoints
- Prevent cross-user data access
- Add environment variables
- Apply to 5–10 roles

Day 28 — CRUD Quality

- Improve create/read/update/delete flows
- Validation and consistent errors
- Add tests for core CRUD
- Apply to 5–10 roles

Day 29 — Filtering

- Filter by category/date/amount
- Query parameters
- Test edge cases
- Apply to 5–10 roles

Day 30 — Pagination

- Implement pagination
- Test limits/offsets
- Review SQL query efficiency basics
- Apply to 5–10 roles

Day 31 — Categories + Budgets

- Category management

- Budget model
- Budget endpoints or basic budget logic
- Apply to 5–10 roles

Day 32 — Analytics

- Monthly spending summary
- Category spending summary
- Total spending
- Test analytics endpoints
- Apply to 5–10 roles

Day 33 — Pytest

- Set up pytest
- Test registration/login
- Test CRUD
- Test unauthorized access
- Run full test suite
- Apply to 5–10 roles

Day 34 — Docker

- Write Dockerfile
- Docker Compose basics
- Containerize API + PostgreSQL
- Environment variables
- Run locally in containers
- Apply to 5–10 roles

Day 35 — Project 1 Freeze

- Clean code structure
- README with architecture/features/setup
- API endpoint documentation
- Meaningful Git commits
- Prepare 5-minute project explanation
- Apply to 5–10 roles

Day 36 — Deployment

- Choose a practical deployment path
- Deploy backend if feasible
- Verify production configuration
- Update README
- Apply to 5–10 roles

Day 37 — Interview Python I

- Revise lists/dicts/sets/tuples
- Mutability

- Functions
- Solve 5 Python problems
- Apply to 5–10 roles

Day 38 — Interview Python II

- OOP
- Exceptions
- Comprehensions
- Iterators/generators basics
- Solve 5 Python problems
- Apply to 5–10 roles

Day 39 — Interview SQL

- Joins
- GROUP BY/HAVING
- Subqueries
- PK/FK/indexes
- Solve 10 SQL questions
- Apply to 5–10 roles

Day 40 — Project 1 Mock Interview

- Explain architecture
- Explain request flow
- Explain JWT
- Explain database design
- Explain testing/Docker
- Record a 10-minute explanation
- Apply to 5–10 roles

Day 41 — Project 2: PDF Processing

- Set up Resume Analyzer repository
- Learn PyMuPDF
- Extract text from PDF
- Clean extracted text
- Create upload endpoint
- Apply to 5–10 roles

Day 42 — Resume Parsing

- Handle multiple PDF cases
- Normalize text
- Store resume metadata
- Test extraction
- Apply to 5–10 roles

Day 43 — NLP Basics

- Text normalization
- Tokenization concepts
- Stop words concepts
- TF-IDF concept
- Implement a small TF-IDF experiment
- Apply to 5–10 roles

Day 44 — Similarity

- Cosine similarity
- Compare resume vs job description
- Return similarity score
- Test with sample jobs
- Apply to 5–10 roles

Day 45 — Skill Extraction

- Create normalized skill dictionary
- Extract skills from resume
- Extract skills from job description
- Return matched/missing skills
- Apply to 5–10 roles

Day 46 — Skill Matching API

- Create analysis endpoint
- Persist analysis results
- Add Pydantic schemas
- Test endpoints
- Apply to 5–10 roles

Day 47 — Semantic Matching

- Learn embeddings concept
- Generate embeddings using a suitable library/API
- Compare semantic similarity
- Document approach
- Apply to 5–10 roles

Day 48 — Matching Evaluation

- Create sample resume/job pairs
- Compare TF-IDF vs semantic matching
- Inspect failure cases
- Improve normalization
- Apply to 5–10 roles

Day 49 — LLM Feature

- Add LLM integration
- Generate resume improvement suggestions

- Handle API keys securely
- Add error handling
- Apply to 5–10 roles

Day 50 — Skill Gap Recommendations

- Generate skill-gap explanation
- Job-specific recommendations
- Keep recommendations grounded in extracted data
- Test outputs
- Apply to 5–10 roles

Day 51 — Project 2 Backend

- Complete FastAPI routes
- PostgreSQL persistence
- Validation
- Error handling
- Write core tests
- Apply to 5–10 roles

Day 52 — React Dashboard

- Create dashboard shell
- Resume upload UI
- Job description input
- Connect frontend to API
- Apply to 5–10 roles

Day 53 — Results UI

- Match score
- Matched skills
- Missing skills
- Recommendations
- Handle loading/errors
- Apply to 5–10 roles

Day 54 — Docker + Polish

- Dockerize where practical
- Environment configuration
- Clean README
- Screenshots/demo notes
- Apply to 5–10 roles

Day 55 — Project 2 Freeze

- Test complete flow
- Fix major bugs
- Create meaningful Git commits

- Prepare 5-minute explanation
- Add project to resume/portfolio
- Apply to 5–10 roles

Day 56 — Python Interview Day

- Revise core Python
- OOP
- Exceptions
- Comprehensions
- Generators/decorators basics
- Solve 5 coding problems
- Apply to 5–10 roles

Day 57 — SQL + Backend Interview

- Revise SQL
- HTTP/REST
- FastAPI
- Pydantic
- SQLAlchemy
- Solve 10 mixed questions
- Apply to 5–10 roles

Day 58 — Authentication + System Flow

- Explain JWT
- Authentication vs authorization
- Explain API request lifecycle
- Explain database sessions
- Explain Docker
- Apply to 5–10 roles

Day 59 — Project Interview

- Explain Expense Tracker for 10–15 minutes
- Explain Resume Analyzer for 10–15 minutes
- Prepare trade-offs
- Prepare failure/bug stories
- Apply to 5–10 roles

Day 60 — Full Mock Interview + Review

- Complete Python round
- Complete SQL round
- Complete backend round
- Complete project round
- Complete HR round
- List top 5 skill gaps

■ Set next 30-day learning targets

■ Continue applications